EXPERIMENT NO. 6

LO1: Study and Implementation of Agile Software Development Lifecycle using Jira with DevOps Pipeline Integration

Aim

To study the Agile Software Development Lifecycle (SDLC) and implement it practically using Jira for Scrum-based project management, integrated with a GitHub repository and a DevOps CI/CD pipeline built using GitHub Actions.

Software / Tools Used

- Jira Cloud (Atlassian) — Agile project management tool
- GitHub — version control and source code repository
- GitHub Actions — CI/CD pipeline automation
- Git (command line) — local version control

Theory

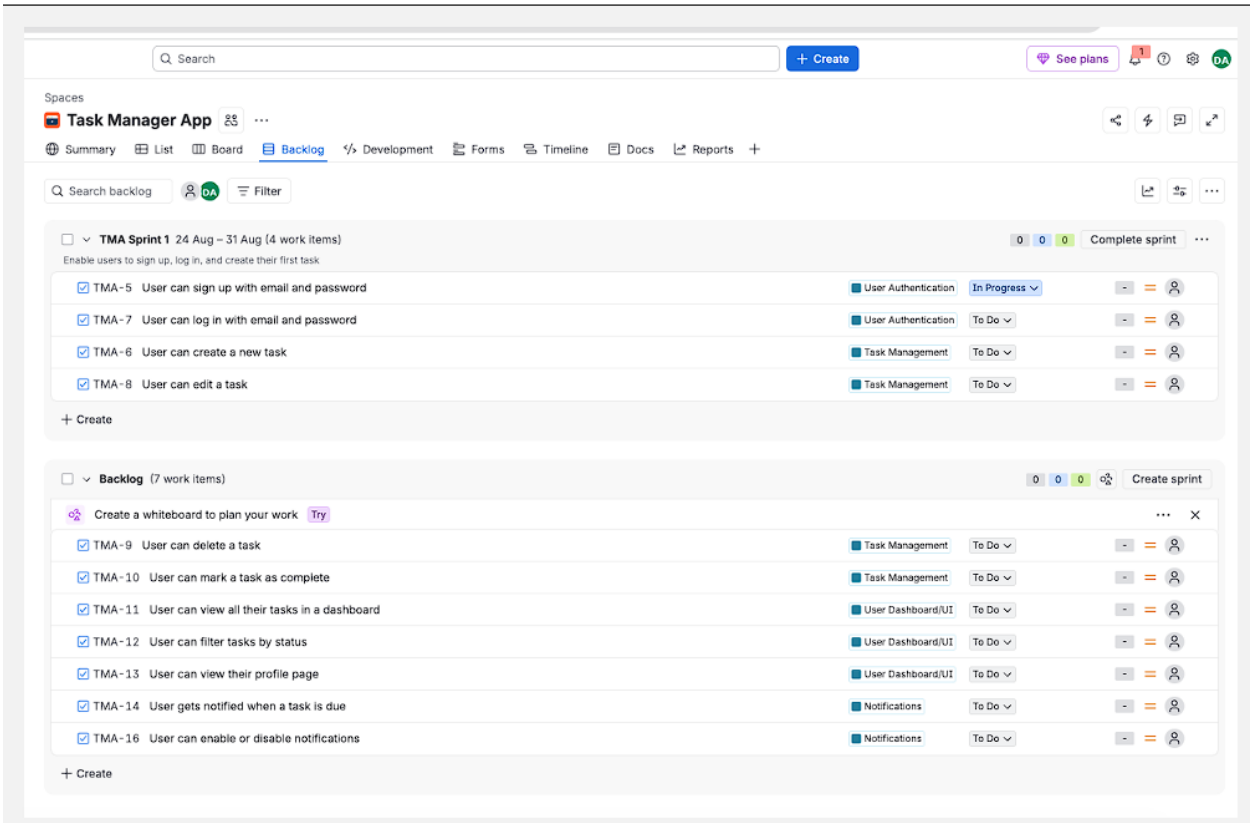
Agile is an iterative approach to software development that emphasizes incremental delivery, collaboration, and flexibility to change. Scrum is a widely used Agile framework that organizes work into fixed-length iterations called Sprints, with work items structured hierarchically as Epics, User Stories, and Tasks.

DevOps extends Agile practices into the deployment pipeline by automating the building, testing, and delivery of code through Continuous Integration and Continuous Deployment (CI/CD). Integrating Jira (planning) with GitHub (source control) and GitHub Actions (automation) creates a complete, traceable pipeline from requirement to deployment.

Procedure

Part A: Jira Project Setup and Backlog Creation

1. Created a Jira Cloud account and set up a new Scrum, team-managed project named “Task Manager App” (project key: TMA).
2. Defined four Epics representing major functional modules: User Authentication, Task Management, User Dashboard/UI, and Notifications.
3. Wrote eleven User Stories in the standard Agile format (“As a user, I want to … so that …”) and linked each to its corresponding Epic.



Part B: Sprint Planning and Execution

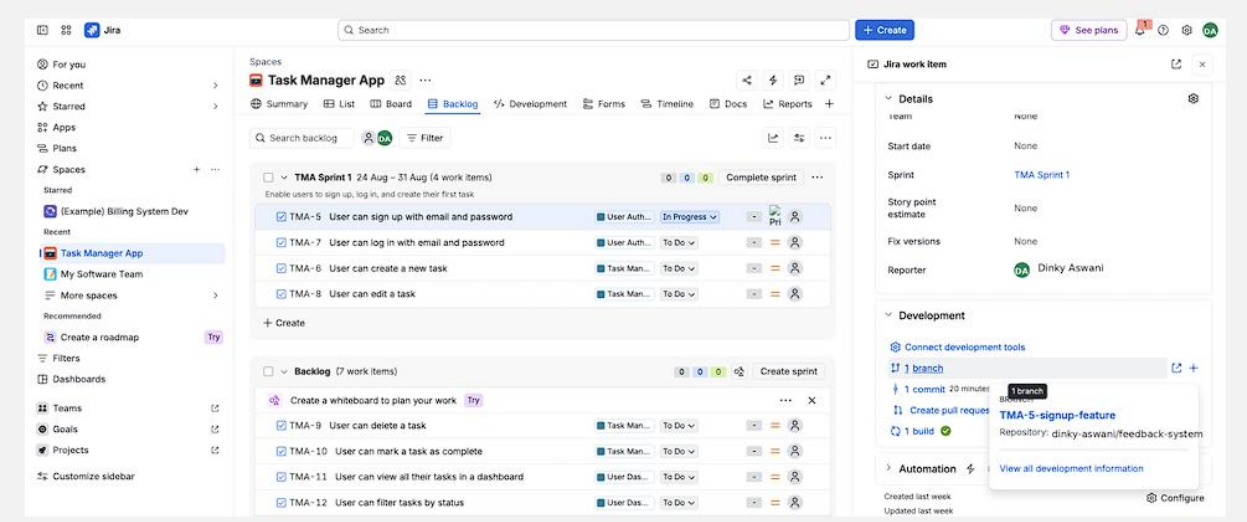
4. Configured Sprint 1 with a one-week duration and a defined Sprint Goal: “Enable users to sign up, log in, and create their first task.”
5. Selected four priority User Stories and moved them into the Sprint.
6. Started the Sprint and used the Scrum board to move a work item through the workflow stages: To Do → In Progress → In Review → Done.



Part C: GitHub Repository and Jira Integration

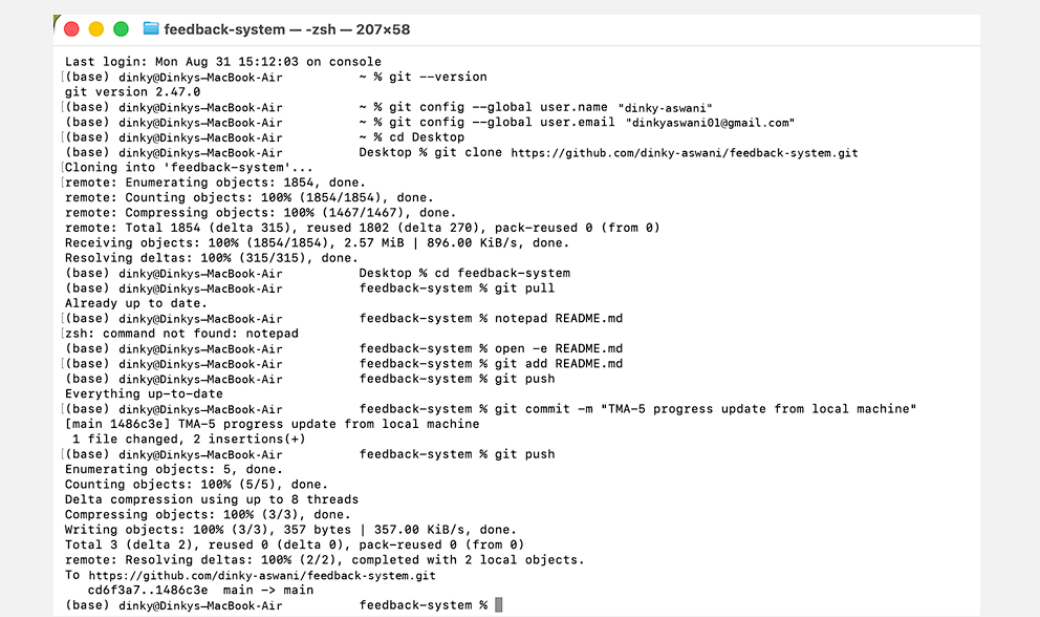
7. Selected an existing GitHub repository (feedback-system) as the codebase for this project.
8. Installed and authorized the official “GitHub for Atlassian” app to connect the GitHub account to the Jira instance.

9. Created a feature branch named TMA-5-signup-feature and made a commit referencing the Jira issue key (TMA-5) in the commit message.
10. Verified that the branch and commit appeared automatically inside the corresponding Jira issue's Development panel.



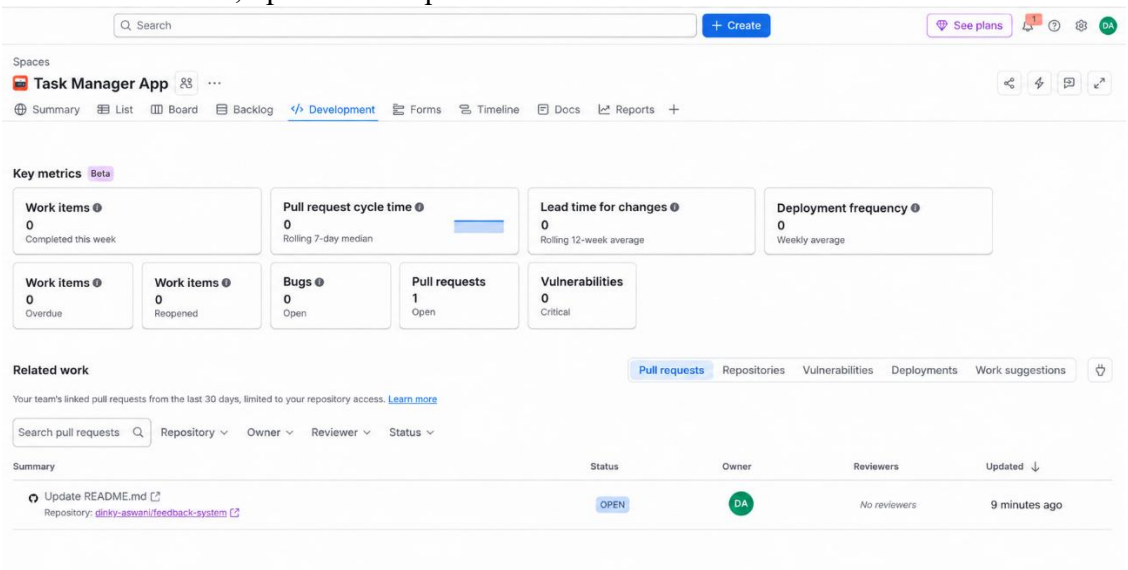
Part D: Local Git Workflow (Push and Pull)

11. Configured Git locally with git config for username and email.
12. Cloned the repository to the local machine using git clone.
13. Executed git pull to fetch the latest changes from the remote repository.
14. Modified a file locally, staged it with git add, committed it referencing TMA-5, and pushed it to GitHub using git push.



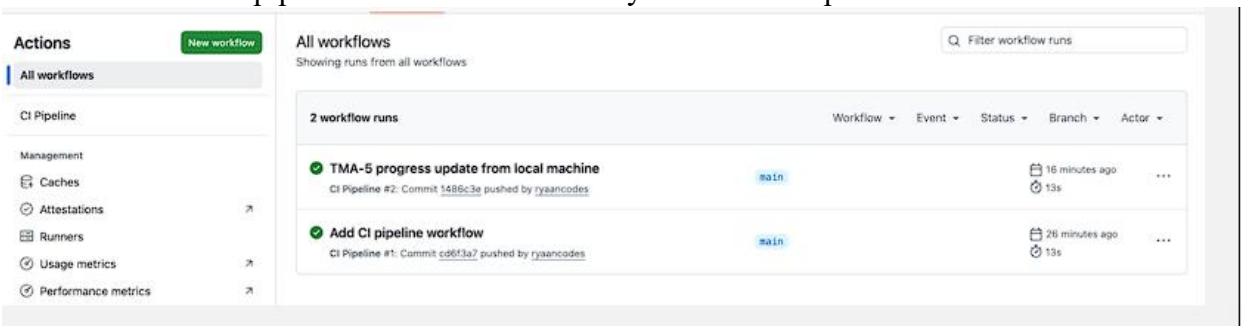
Part E: Pull Request and Jira Development Metrics

15. Opened a Pull Request on GitHub from the feature branch, with the title referencing TMA-5.
16. Confirmed that Jira's project-level Development dashboard automatically updated to show the linked, open Pull Request.



Part F: CI/CD Pipeline using GitHub Actions

17. Created a workflow file at .github/workflows/ci.yml on the main branch.
18. Configured the workflow to trigger on every push to main or any TMA-prefixed branch, performing three steps: checkout code, install dependencies, and run tests.
19. Verified the pipeline executed successfully after the first push.



Result

An Agile Scrum project was successfully implemented in Jira with a structured backlog, active sprint, and board-based tracking. The project was integrated with a GitHub repository,

demonstrating linked branches, commits, and pull requests visible directly within Jira. A DevOps CI/CD pipeline was configured using GitHub Actions and verified to execute successfully on every code push, completing the full loop from Agile planning to automated build and test.

Conclusion

This experiment provided practical understanding of Agile Scrum methodology and its implementation using Jira, along with hands-on experience integrating a project management tool with a version control system and a DevOps CI/CD pipeline. It demonstrated how modern software teams maintain traceability between planning, development, and automated delivery.